🖥️ AWS Compute — Category 2: Complete Interview Guide

> **Target Audience:** DevOps / SRE / Platform / Cloud Engineers
> **Difficulty:** Beginner → Advanced
> **Topics Covered:** EC2, Pricing Models, ASG, Launch Templates, ELB, Placement Groups, IMDSv2, Spot Instances

---

📋 Table of Contents

1. 2.1 EC2 — Instance Types, AMIs, Key Pairs, User Data
2. 2.2 EC2 Pricing Models
3. 2.3 Auto Scaling Groups (ASG)
4. 2.4 Launch Templates vs Launch Configurations
5. 2.5 Elastic Load Balancing — ALB, NLB, CLB, GWLB
6. 2.6 EC2 Placement Groups
7. 2.7 EC2 Instance Metadata & IMDSv2
8. 2.8 Spot Instances — Interruption Handling, Spot Fleets, Diversification

---

2.1 EC2 — Instance Types, AMIs, Key Pairs, User Data

🟢 What It Is in Simple Terms

EC2 (Elastic Compute Cloud) is AWS's virtual machine service. You rent a computer in AWS's data center, choose its size, operating system, and software, and it's ready in minutes. Everything else in AWS — containers, databases, load balancers — either runs ON EC2 or was built to replace it.

🔍 Why It Exists / What Problem It Solves

Before cloud, companies bought physical servers. That meant:

- 6–12 week lead times to provision hardware
- Paying for peak capacity even during off-peak hours
- No way to scale instantly during traffic spikes

EC2 solved this by virtualizing compute — you get a slice of a physical host, billed by the second, disposable at will.

⚙️ How It Works Internally

```
Physical Host (Nitro Hypervisor)
├── EC2 Instance A (your VM)
├── EC2 Instance B (another customer's VM)
└── EC2 Instance C (another customer's VM)

AWS Nitro System:
┌──────────────────────────────────────┐
│         Nitro Hypervisor             │  ← Lightweight, near bare-metal
perf
│   ┌──────────┐   ┌──────────────────┐ │
│   │ Nitro    │   │ Nitro Security   │ │
│   │ Cards    │   │ Chip             │ │
│   │(Network/ │   │(Hardware root of │ │
│   │ Storage) │   │ trust)           │ │
│   └──────────┘   └──────────────────┘ │
└──────────────────────────────────────┘
```

AWS uses the **Nitro hypervisor** — it offloads network and storage virtualization to dedicated hardware cards, giving EC2 instances near bare-metal performance. This is why newer instance types (C5, M5, R5+) are faster than older ones (C3, M3).

🧩 Key Components

Instance Type Families

```
Family  Purpose                    Examples
──────────────────────────────────────────────────
t       Burstable (dev/test)       t3.micro, t3.medium
m       General purpose            m6i.large, m7g.xlarge
c       Compute optimized          c6i.2xlarge, c7g.4xlarge
r       Memory optimized           r6i.8xlarge, r7g.16xlarge
i       Storage optimized (NVMe)   i3.xlarge, i4i.2xlarge
p/g     GPU (ML/graphics)          p4d.24xlarge, g5.xlarge
hpc     High perf computing        hpc6a.48xlarge
```

Instance Size Naming Convention

```
    m   6   i  .  2  x  large
    │   │   │     │  │    │
    │   │   │     │  └────┴── Size modifier
    │   │   │     └──────────── # of units (1x, 2x, 4x...)
    │   │   └─────────────────── Processor (i=Intel, a=AMD, g=Graviton)
    │   └─────────────────────── Generation (higher = newer/better)
    └─────────────────────────── Family (m=general purpose)
```

> ⚠ **Interview Gotcha:** `t3` instances use **CPU Credits**. They earn credits when idle and spend them when bursting. If you run out of credits in `standard` mode, performance drops. In `unlimited` mode you get charged extra. This trips up many candidates.

AMIs (Amazon Machine Images)

An AMI is a snapshot of an OS + software configuration used to launch EC2 instances.

```
AMI contains:
├── Root volume snapshot (OS, installed packages)
├── Block device mappings (which EBS volumes to attach)
├── Launch permissions (who can use this AMI)
└── Architecture (x86_64 or arm64)

AMI Types:
├── AWS-provided    → Amazon Linux 2, Ubuntu, Windows Server
├── AWS Marketplace → Pre-configured (e.g., Nginx Plus, Splunk)
├── Community       → Shared by other AWS users (use with caution)
└── Your own        → Golden AMIs baked by your team
```

Golden AMI Pattern (very common in production):

```
Base OS → Install agents (CW, SSM, etc.)
       → Harden (CIS benchmarks)
       → Run Packer
       → Output: Your Golden AMI
       → ASG uses this AMI for all instances
```

Key Pairs

```
How SSH key pairs work with EC2:

Your machine          AWS
──────────            ──────────────────────────
Private Key    ←→     Public Key stored in ~/.ssh/authorized_keys
(you keep it)         (injected at boot via cloud-init)

ssh -i my-key.pem ec2-user@<public-ip>
```

> ⚠ **Gotcha:** AWS stores only the public key. If you lose your private key, you CANNOT recover it. For prod, most teams use **SSM Session Manager** instead of SSH — no key pair needed, no open port 22.

User Data

Script that runs **once** at first boot (via cloud-init):

```
#!/bin/bash
# This runs as root at instance launch
yum update -y
yum install -y nginx
systemctl enable nginx
systemctl start nginx
echo "Hello from $(hostname)" > /var/www/html/index.html
```

```
Instance Launch Timeline:
0s   → Hypervisor allocates VM
~10s → cloud-init starts
~15s → User data script executes
~60s → Instance health checks pass
~90s → Instance "InService" in ASG
```

> ⚠ **Gotcha:** User data runs as **root**. It runs only on first boot by default. If your script fails, the instance still launches — EC2 doesn't roll back. Debugging: check `/var/log/cloud-init-output.log`

EC2 Status Checks — System vs Instance

```
AWS runs two automated health checks on every EC2 instance
every minute. They are SEPARATE checks testing DIFFERENT layers.

┌──────────────────────────────────────────────────────────────┐
│              EC2 Status Check Architecture                   │
│                                                              │
│  Physical Host                                               │
│  ┌──────────────────────────────────────────────────────┐   │
│  │  System Status Check  ← tests THIS layer             │   │
│  │  (AWS infrastructure: power, network, hardware, host OS) │   │
│  │                                                      │   │
│  │    ┌──────────────────────────────────────┐          │   │
│  │    │  Your EC2 Instance                   │          │   │
│  │    │  Instance Status Check  ← tests THIS layer      │   │
│  │    │  (your VM: kernel, OS, network config, memory)  │   │
│  │    └──────────────────────────────────────┘          │   │
│  └──────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────┘
```

```
SYSTEM STATUS CHECK
────────────────────
What it tests:   AWS infrastructure BELOW your instance
                 ├── Physical host hardware (CPU, memory, disk)
                 ├── Host power supply
                 ├── Physical network connectivity to the host
                 └── AWS hypervisor (Nitro) health

Failure means:   AWS's hardware/infrastructure has a problem
                 NOT your fault — AWS must fix it

Fix options:
├── Wait: AWS detects and migrates/repairs automatically (can take time)
├── Stop + Start: instance moves to a DIFFERENT physical host
│   ⚠ Stop + Start, NOT reboot — reboot stays on the same host
│   ⚠ Public IP changes on Stop + Start unless using Elastic IP
└── Auto-recovery: CloudWatch alarm → recover action (see below)

CloudWatch metric: StatusCheckFailed_System
```

```
INSTANCE STATUS CHECK
──────────────────────
What it tests:   YOUR instance's software and OS
                 ├── OS kernel — is it running and responding?
                 ├── Network interface config (incorrect IP, routing)
                 ├── System memory exhaustion (OOM killer)
                 ├── File system corruption or full disk
                 └── Incompatible kernel modules / bad startup scripts

Failure means:   Something inside YOUR instance is broken
                 AWS infrastructure is fine — you must fix it

Fix options:
├── Reboot: restarts the OS (stays on same host — fine here)
├── SSH in and fix: check /var/log/messages, dmesg, disk space
├── Stop + Start with modified user data: fix startup script
└── Restore from snapshot: if OS corruption is unrecoverable

CloudWatch metric: StatusCheckFailed_Instance
```

```
Combined metric:
StatusCheckFailed = 1 if EITHER check fails (System OR Instance)
                    0 if both checks pass

Check status in console:
EC2 → Instances → select instance → Status Checks tab
Shows: System reachability + Instance reachability + pass/fail
```

```
# Query status check results via CLI
aws ec2 describe-instance-status \
  --instance-ids i-0abc123def456789 \
  --query 'InstanceStatuses[*].{
    InstanceId:InstanceId,
    System:SystemStatus.Status,
    Instance:InstanceStatus.Status
  }'

# Output:
# [
#   {
#     "InstanceId": "i-0abc123def456789",
#     "System":   "ok",       ← or "impaired"
#     "Instance": "impaired"  ← OS-level problem
#   }
# ]
```

Auto-Recovery — CloudWatch Alarm Action

```
Auto-recovery = CloudWatch alarm watches StatusCheckFailed_System
                When it fires → AWS automatically recovers the instance

What "recover" does:
├── Moves instance to a new healthy physical host
├── Preserves: instance ID, private IP, Elastic IP, EBS volumes
├── Loses:     instance store data (ephemeral storage only)
│              RAM contents
└── Takes:     ~5-10 minutes typically

⚠ Auto-recovery works ONLY for:
   ├── System status check failures (not instance status check)
   ├── EBS-backed instances (not instance store root)
   └── Supported instance types (most modern types — check AWS docs)
```

```
# Create CloudWatch alarm with auto-recovery action
aws cloudwatch put-metric-alarm \
  --alarm-name "auto-recover-i-0abc123def456789" \
  --alarm-description "Auto recover EC2 on system status check failure" \
  --namespace AWS/EC2 \
  --metric-name StatusCheckFailed_System \
  --dimensions Name=InstanceId,Value=i-0abc123def456789 \
  --statistic Minimum \
  --period 60 \
  --evaluation-periods 2 \
  --threshold 1 \
  --comparison-operator GreaterThanOrEqualToThreshold \
```

```
  --alarm-actions \
    arn:aws:automate:us-east-1:ec2:recover \
    arn:aws:sns:us-east-1:123456789:ops-alerts
  # Two actions: recover the instance AND notify the team

# Other available alarm actions for EC2:
# arn:aws:automate:region:ec2:recover   → recover (new host, same config)
# arn:aws:automate:region:ec2:reboot    → reboot the instance
# arn:aws:automate:region:ec2:stop      → stop the instance
# arn:aws:automate:region:ec2:terminate → terminate the instance
```

How Status Checks Connect to ASG

```
ASG health check type — this is where status checks matter most:

ASG health check type = EC2 (default):
├── ASG marks instance unhealthy if:
│   EC2 instance state is NOT "running"
│   (stopped, terminated, shutting-down)
├── Does NOT consider: StatusCheckFailed_Instance
│   → Your OS can be completely broken, ASG thinks it's healthy!
└── Use for: simple stateless apps where OS health = VM running

ASG health check type = ELB:
├── ASG marks instance unhealthy if:
│   Load balancer health check fails (your /health endpoint returns non-
2xx)
├── ALSO considers EC2 state (both must pass)
└── Use for: production web apps — tests actual application health

ASG health check type = EC2 + custom:
├── You can combine: if StatusCheckFailed → ASG replaces instance
└── Pattern: CloudWatch alarm on StatusCheckFailed → SNS → Lambda
             Lambda calls SetInstanceHealth to mark it unhealthy
             ASG replaces it automatically

aws autoscaling set-instance-health \
  --instance-id i-0abc123def456789 \
  --health-status Unhealthy
# ASG will terminate and replace this instance
```

Status Check Failure Diagnosis Cheatsheet

```
Symptom                          | Which check  | Likely cause
---------------------------------|--------------|---------------------------
—
Instance unreachable, AWS issue  | System       | Host hardware failure
```

```
Instance running, SSH fails      | Instance     | OS crash / kernel panic
High CPU but instance check OK   | Neither      | App-level issue (not
checks)
Instance check fails after deploy| Instance     | Bad startup script / OOM
Intermittent check failures      | System       | Degraded host —
stop+start
Disk full                        | Instance     | OS can't function →
fails
Network misconfigured            | Instance     | Wrong IP / routing in OS
Both checks fail                 | Both         | Stop+start first, then
debug
```

Status Checks vs ELB Health Checks vs ASG Health Checks

```
Three different health check systems — often confused in interviews:

┌──────────────────────┬──────────────────────────┬──────────────────┐
│ Check Type           │ What It Tests            │ Who Acts On It   │
├──────────────────────┼──────────────────────────┼──────────────────┤
│ EC2 Status Check     │ Hardware + OS layer      │ CloudWatch alarm
│
│ (System + Instance)  │ "Is the VM alive?"       │ → auto-recover
│
├──────────────────────┼──────────────────────────┼──────────────────┤
│ ELB Health Check     │ Application layer        │ Load balancer
│
│ (Target Group)       │ "Is /health returning 200?"  │ → stops routing
│
├──────────────────────┼──────────────────────────┼──────────────────┤
│ ASG Health Check     │ EC2 state OR ELB check   │ ASG
│
│                      │ "Should I replace this?" │ → terminates +
│
│                      │                          │   launches new
│
└──────────────────────┴──────────────────────────┴──────────────────┘

The full production stack uses ALL THREE:
EC2 Status Check  → CloudWatch alarm → auto-recover (hardware failures)
ELB Health Check  → stops sending traffic (app failures)
ASG Health Check  → replaces the instance (persistent failures)
```

💬 Short Crisp Interview Answer

"EC2 is AWS's virtual machine service built on the Nitro hypervisor. You choose an instance family based on your workload — compute, memory, storage, or GPU optimized — and a size. AMIs are the blueprint — they contain the OS and pre-installed software. User data lets you bootstrap instances on first boot. In

production, we typically build golden AMIs with our agents pre-installed and use SSM Session Manager instead of key pairs for access."

🔬 Deep Dive Version

"EC2 runs on AWS's Nitro system which offloads network and storage I/O to dedicated hardware cards, giving near bare-metal performance. Instance types follow a naming convention — family, generation, processor variant, size. t-family instances are special — they use a CPU credit model for bursting. AMIs are region-specific but can be copied cross-region. For production, the golden AMI pattern with Packer is standard — you bake all agents, hardening, and config into the AMI so instances are immutable and launch fast. User data is a one-time bootstrap script but for idempotent configuration management, you'd layer in SSM State Manager or Ansible on top."

🏭 Real World Production Example

```
Team: Platform Engineering at a fintech company

Problem: Dev teams spinning up EC2s with inconsistent configs,
         missing security agents, different OS versions.

Solution: Golden AMI Pipeline

   ┌──────────┐     ┌──────────┐     ┌──────────────┐     ┌──────────┐
   │ Base AMI │──▶│ Packer   │──▶│ Golden AMI   │──▶│  ASG     │
   │(AmazonLX)│     │ Script   │     │(hardened+    │     │Launch    │
   └──────────┘     └──────────┘     │ agents baked)│     │Template) │
                                     └──────────────┘     └──────────┘

Packer script installs:
- CloudWatch Agent
- SSM Agent
- Datadog Agent
- CIS hardening scripts
- Company CA certificates

Result:
- All instances identical at launch
- No SSH, no key pairs — SSM only
- AMI rebuilt weekly with latest patches
- New AMI triggers ASG instance refresh
```

❓ Common Interview Questions & Strong Answers

Q: What's the difference between stopping and terminating an EC2 instance?

> Stopping preserves the EBS root volume and the instance's private IP (in a VPC). RAM is lost, the host may change. Terminating deletes the instance and, by default, deletes the root EBS volume. The

> key gotcha: if `DeleteOnTermination` is false for the root volume, the EBS persists and you keep paying.

Q: Can you change the instance type of a running EC2?

> No — you must stop it first, then change the instance type, then start it. The instance may land on a different physical host. The private IP stays the same (VPC), but the public IP changes unless you're using an Elastic IP.

Q: How does EC2 instance metadata work?

> Every instance can query `http://169.254.169.254/latest/meta-data/` to get its own metadata — instance ID, AZ, IAM role credentials, public IP, etc. IMDSv2 is the secure version — it requires a session token obtained via a PUT request first, preventing SSRF attacks from stealing credentials.

CPU Steal Time — the hidden performance thief

```
CPU steal time = time your vCPU WANTS to run but the hypervisor
                 has given the physical CPU to another tenant's VM

Your instance: "I need CPU right now"
Hypervisor:    "Wait — another VM is using that core"
Result:        Your process stalls, even though your vCPU shows as busy

Linux sees this as: %steal in top / vmstat output

  top output:
  %Cpu(s): 12.3 us, 2.1 sy, 0.0 ni, 71.4 id, 3.2 wa, 0.0 hi, 0.0 si, 11.0
st

^^^^
                                                                    steal
= 11%!

  Healthy steal:  < 5%
  Concerning:     5-10% (investigate, consider upgrade)
  Critical:       > 10% (your workload is being starved)
```

```
Why steal time happens:
├── You share a physical host with other tenants (noisy neighbor problem)
├── Over-provisioned physical host → more vCPUs promised than physical
cores
├── t-family burstable instances are especially prone under sustained load
└── Nitro instances (5th gen+) have lower steal than older Xen-based
instances

Why Nitro reduced steal time:
├── Network and storage I/O offloaded to dedicated Nitro cards
```

```
│   → Nitro card handles EBS, VPC networking independently
│   → Physical host CPUs are free for your workload exclusively
├── Less hypervisor overhead competing for the same physical cores
└── This is one of the main reasons to use m6i/c6i over m4/c4
```

```
# How to measure steal time on a running instance

# Method 1: top (column 'st')
top
# Press '1' to see per-CPU steal

# Method 2: vmstat (st column, reported per second)
vmstat 1 10
# procs -----------memory---------- ---swap-- -----io---- -system-- ------
cpu-----
# r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy
id wa st
# 2  0      0 1234567  12345 234567    0    0     0     5  123  456  8  2
79  0  11

# Method 3: sar (historical steal data)
sar -u 1 10   # -u = CPU, 1 second interval, 10 samples
# %steal column shows historical pattern

# Method 4: CloudWatch custom metric (push steal time — not collected
natively)
STEAL=$(vmstat 1 2 | tail -1 | awk '{print $16}')
aws cloudwatch put-metric-data \
  --namespace "EC2/Custom" \
  --metric-name "CPUStealTime" \
  --value "$STEAL" \
  --unit Percent \
  --dimensions InstanceId=$(curl -sf \
    -H "X-aws-ec2-metadata-token: $TOKEN" \
    http://169.254.169.254/latest/meta-data/instance-id)
```

```
What to do when steal time is high:

Option 1: Move to a larger instance type
          Larger instances → fewer tenants per physical host
          → less contention for physical cores

Option 2: Move to a newer generation (Nitro)
          m4 → m6i, c4 → c6i, r4 → r6i
          Nitro offloads I/O → less host CPU competition

Option 3: Use Dedicated Instance or Dedicated Host
          Dedicated Instance: your VMs on physically isolated hardware
                              no other AWS customer shares your host
```

```
            Dedicated Host:     you choose which physical host your VMs land
on
                                useful for BYOL (Bring Your Own License)
            ⚠ Dedicated options cost significantly more (~10-20% premium)

Option 4: Placement Groups do NOT fix steal time
          Cluster PG = network proximity, not CPU isolation
          Dedicated Instance/Host is the correct fix for steal

Option 5: Accept it for t-family instances
          t3/t4g are burstable — designed for bursty workloads
          Sustained high steal on t3 → wrong instance family
          Move to m or c family for consistent compute needs
```

```
Steal time vs other CPU metrics — how to tell them apart:

High us (user):   your application code is CPU-bound → optimize or scale
out
High sy (system): kernel overhead → check I/O, syscalls, context switches
High wa (wait):   waiting on I/O → EBS throughput or IOPS bottleneck
High st (steal):  hypervisor gave your CPU to someone else → host problem
High id (idle):   spare capacity → no issue

steal is the ONLY metric not caused by your own workload.
It is an infrastructure-layer problem — optimizing your code does nothing.
The fix is always at the instance/host level.
```

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
t3 CPU credits	Standard mode = throttled when credits run out. Unlimited = surprise bill
AMI is region-locked	You must copy AMI to use in another region
User data size limit	16 KB max. Use S3 + curl for larger scripts
Key pairs ≠ IAM	SSH key pair controls OS access, IAM role controls AWS API access. They're separate
Stopped instance billing	You don't pay for compute when stopped, but you still pay for EBS and Elastic IPs

Gotcha	Detail
Instance store	Some instance types have ephemeral local NVMe. Data is LOST on stop/terminate
CPU steal time	Not in CloudWatch by default — must push as custom metric. >10% steal = noisy neighbor. Fix: larger instance, newer Nitro gen, or Dedicated Instance

🔗 Connections to Other AWS Concepts

```
EC2 connects to:
├── IAM          → Instance Profiles give EC2 AWS API permissions
├── VPC          → EC2 lives in a subnet, gets a security group
├── EBS          → Block storage attached to EC2
├── ASG          → EC2 fleet management, scaling
├── ALB/NLB      → Traffic distribution to EC2 instances
├── SSM          → Remote access, patching, config management
├── CloudWatch   → Metrics, logs from EC2
├── AMI          → Blueprint for EC2
└── Nitro        → Underlying hypervisor giving performance
```

2.2 EC2 Pricing Models

🟢 What It Is in Simple Terms

AWS gives you 4 ways to pay for EC2. Pick the wrong one and you'll massively overpay. Pick the right one and you can cut your compute bill by 70–90%.

⚙️ How Each Model Works

```
┌──────────────────┬──────────┬──────────────┬──────────────────────┐
│ Model            │ Discount │ Commitment   │ Best For             │
├──────────────────┼──────────┼──────────────┼──────────────────────┤
│ On-Demand        │ 0%       │ None         │ Unpredictable workloads│
│ Reserved (1yr)   │ ~40%     │ 1 year       │ Steady-state baseline │
│ Reserved (3yr)   │ ~60%     │ 3 years      │ Long-term baseline    │
│ Savings Plans    │ ~66%     │ 1 or 3 years │ Flexible baseline     │
│ Spot             │ ~70-90%  │ None         │ Fault-tolerant/batch  │
└──────────────────┴──────────┴──────────────┴──────────────────────┘
```

On-Demand

- Pay by the second (minimum 60 seconds)
- No commitment, highest price
- Use for: spiky traffic, dev/test, new applications you can't predict

---

Reserved Instances (RIs)

```
3 payment options:
├── All Upfront     → Biggest discount, pay everything now
├── Partial Upfront → Pay some now, rest monthly
└── No Upfront      → Pay monthly, smallest discount

2 scope options:
├── Regional RI → Applies to any AZ in the region (flexible)
└── Zonal RI    → Applies to specific AZ (+ capacity reservation)

Standard RI    → Cannot change instance family (m5 stays m5)
Convertible RI → Can exchange for different family/OS (more flexible, less
discount)
```

> ⚠ **Gotcha:** RIs don't "attach" to instances. They're a billing construct. AWS automatically applies RI discounts to matching running instances. You can have 10 RIs and 0 running instances — you're still paying for the RIs.

Savings Plans

Newer, more flexible alternative to RIs:

```
Compute Savings Plan:
└── Commit to $/hour spend (e.g., $10/hr)
    └── Applies to ANY EC2 instance family, region, size, OS
    └── Also applies to Fargate and Lambda
    └── Most flexible — recommended over RIs for most teams

EC2 Instance Savings Plan:
└── Commit to specific instance family in a region (e.g., m5 in us-east-1)
    └── Higher discount than Compute SP (~66% vs ~60%)
    └── Can change size, AZ, OS within family
```

Spot Instances

```
How Spot pricing works:

AWS has spare capacity → offers it at 70-90% discount
Your Spot instance runs → until AWS needs capacity back
```

```
AWS needs capacity     → 2-minute warning then termination

Spot Instance States:
open → running → interrupted (2-min notice) →
terminated/stopped/hibernated
```

💬 Short Crisp Interview Answer

"AWS has four EC2 pricing models. On-Demand is full price, no commitment — good for unpredictable workloads. Reserved Instances or Savings Plans give 40–66% off in exchange for a 1 or 3 year commitment — you use these for your steady-state baseline. Spot instances use spare AWS capacity at 70–90% off but can be interrupted with 2 minutes notice — ideal for batch jobs, stateless workers, and fault-tolerant workloads. In production, a typical strategy is: Savings Plans for baseline, On-Demand for buffer, and Spot for burst or batch."

🏭 Real World Production Example

```
Strategy: "3-tier compute cost optimization"

Your workload: Web app, always needs minimum 10 instances,
               sometimes needs up to 50 during business hours,
               runs nightly batch jobs.

Solution:
┌──────────────────────────────────────────────────────────┐
│  10 instances  → Compute Savings Plan (always on)        │
│  +15 instances → On-Demand (business hour buffer)        │
│  +25 instances → Spot (nightly batch processing)         │
└──────────────────────────────────────────────────────────┘

Cost savings vs all On-Demand:
- Savings Plan: saves ~60% on 10 instances
- Spot batch: saves ~80% on nightly jobs
- Net result: 40-50% overall bill reduction
```

❓ Common Interview Questions

Q: What happens to your Spot instance when AWS interrupts it?

> You get a 2-minute warning via instance metadata at `latest/meta-data/spot/termination-time` and via EventBridge. Your app should handle this gracefully — checkpoint state, drain connections, deregister from load balancer. ASGs with mixed instance policies handle this automatically.

Q: Can you use Reserved Instances across accounts?

> Yes — in AWS Organizations, RIs and Savings Plans can be shared across accounts via the management account's billing settings. You can also sell unused Standard RIs on the AWS Marketplace.

Q: Savings Plans vs Reserved Instances — which do you recommend?

> Compute Savings Plans in almost all cases. They're more flexible — they apply across families, regions, and even Fargate/Lambda. The only reason to choose RIs is if you need capacity reservations in a specific AZ (zonal RIs), or if you're on a very specific instance family long-term and want maximum discount.

⚠️ Gotchas

Gotcha	Detail
RI billing trap	Unused RIs still cost money — you pay whether or not you run instances
Spot + stateful apps	Never run databases or stateful apps on Spot without interruption handling
Savings Plans stack	Your SP covers usage up to committed amount; excess is On-Demand
Convertible RI exchange	You can exchange but the new RI must be equal or greater value
Spot price ≠ bid price anymore	AWS moved to market pricing — you pay current Spot price, not your bid

---

2.3 Auto Scaling Groups (ASG)

🟢 What It Is in Simple Terms

An ASG is a fleet manager for EC2 instances. You tell it: "I always want between 2 and 20 instances, target 70% CPU, add/remove instances automatically." ASG handles the rest — launching, terminating, replacing unhealthy instances, and integrating with load balancers.

⚙️ How It Works Internally

```
ASG Architecture:

                    ┌──────────────────────────┐
                    │    Auto Scaling Group    │
                    │  Min: 2  Max: 20         │
                    │  Desired: 5              │
                    └──────────────────────────┘
                                │ manages
                ┌───────────────┼──────────────┐
```

```
                ▼                ▼                ▼
          ┌─────────┐        ┌─────────┐        ┌─────────┐
          │  EC2    │        │  EC2    │        │  EC2    │
          │  i-1    │        │  i-2    │        │  i-3    │
          │  AZ-a   │        │  AZ-b   │        │  AZ-c   │
          └─────────┘        └─────────┘        └─────────┘

ASG Health Check Loop (every 5 min default):
┌──────────────────────────────────────────────────┐
│ 1. Check each instance (EC2 or ELB health)       │
│ 2. If unhealthy → terminate → launch new         │
│ 3. Check scaling policies                        │
│ 4. If scale-out triggered → launch instances     │
│ 5. If scale-in triggered → terminate (oldest     │
│    launch config OR closest to billing hour)     │
└──────────────────────────────────────────────────┘
```

🧩 Key Components

Scaling Policies

```
3 Types of Scaling Policies:

1. TARGET TRACKING (recommended, simplest)
   ──────────────────────────────────────────
   "Keep average CPU at 50%"
   ASG does the math — adds/removes to hit target
   Built-in metrics: CPU, network, ALB request count per target

2. STEP SCALING (more control)
   ──────────────────────────────────────────
   "If CPU 60-80%  → add 1 instance
    If CPU 80-100% → add 3 instances
    If CPU < 40%   → remove 1 instance"
   Good for non-linear workloads

3. SIMPLE SCALING (legacy, avoid)
   ──────────────────────────────────────────
   "If alarm triggers → add 2, wait cooldown, re-evaluate"
   Slow — waits full cooldown between steps
```

Target Tracking CLI Example:

```
aws autoscaling put-scaling-policy \
  --policy-name cpu-target-tracking \
  --auto-scaling-group-name my-asg \
  --policy-type TargetTrackingScaling \
  --target-tracking-configuration '{
```

```
    "PredefinedMetricSpecification": {
      "PredefinedMetricType": "ASGAverageCPUUtilization"
    },
    "TargetValue": 50.0
  }'
```

Cooldown Periods

```
Problem Cooldown Solves:
New instances launch → metrics look better immediately
→ Without cooldown, ASG would immediately scale back in
→ New instances never finish warming up!

Default Cooldown: 300 seconds (5 min)

Timeline:
0s   → Scale-out triggered (CPU 80%)
0s   → New instance starts launching
90s  → Instance InService
90s  → Cooldown starts (300s)
390s → Cooldown ends → ASG evaluates metrics again

⚠ Key insight: Cooldown applies to Simple Scaling only.
Target Tracking and Step Scaling use "instance warmup"
period instead of cooldown.
```

⚠ Lifecycle Hooks (Frequently Asked)

```
Normal ASG flow:
Pending → InService → Terminating → Terminated

With Lifecycle Hooks:

Scale OUT:
Pending → [Pending:Wait] → [Pending:Proceed] → InService
               │
               └── Your hook fires here!
                   - Register with load balancer
                   - Pull config from S3
                   - Run smoke tests
                   - Max wait: 48 hours

Scale IN:
InService → [Terminating:Wait] → [Terminating:Proceed] → Terminated
                  │
                  └── Your hook fires here!
                      - Drain connections
```

```
                        - Ship final logs
                        - Deregister from service discovery
                        - Take final snapshot

How hooks work:
1. ASG puts instance in Wait state
2. Sends notification to SNS/SQS/EventBridge
3. Your Lambda/script runs
4. Calls CompleteLifecycleAction (CONTINUE or ABANDON)
5. If no response in heartbeat timeout → default action
```

```
# Create a lifecycle hook for scale-in
aws autoscaling put-lifecycle-hook \
  --lifecycle-hook-name drain-connections \
  --auto-scaling-group-name my-asg \
  --lifecycle-transition autoscaling:EC2_INSTANCE_TERMINATING \
  --notification-target-arn arn:aws:sqs:us-east-1:123:my-queue \
  --role-arn arn:aws:iam::123:role/asg-hook-role \
  --heartbeat-timeout 300 \
  --default-result CONTINUE

# Complete lifecycle action from your script
aws autoscaling complete-lifecycle-action \
  --lifecycle-hook-name drain-connections \
  --auto-scaling-group-name my-asg \
  --lifecycle-action-result CONTINUE \
  --instance-id i-1234567890abcdef0
```

Instance Refresh

Used for rolling updates (e.g., deploying a new AMI):

```
aws autoscaling start-instance-refresh \
  --auto-scaling-group-name my-asg \
  --preferences '{
    "MinHealthyPercentage": 80,
    "InstanceWarmup": 120
  }'
```

ASG replaces instances a percentage at a time, ensuring minimum healthy percentage is maintained. Much better than terminate-all-and-relaunch.

💬 Short Crisp Interview Answer

"An ASG manages a fleet of EC2 instances automatically — maintaining desired capacity, replacing unhealthy instances, and scaling in/out based on policies. There are three scaling policy types: target

tracking is simplest and recommended — you set a target metric and ASG does the math. Step scaling gives more control for non-linear loads. Lifecycle hooks are critical in production — they let you pause instance launch or termination to run custom logic like draining connections or registering with service discovery."

🏭 Real World Production Example

```
Problem: Node.js API servers taking 4 minutes to warm up JVM cache.
         ASG was scaling out, declaring instances healthy before warm,
         sending live traffic to cold instances = 500 errors.

Solution: Lifecycle Hook + Custom Warmup

Scale-out lifecycle hook:
1. Instance launches → enters Pending:Wait
2. Hook calls Lambda
3. Lambda polls /health endpoint every 10s
4. When /health returns {"cache": "warm"} →
CompleteLifecycleAction(CONTINUE)
5. Only then does ALB get the instance

Result: Zero 500 errors from cold instances.
        Slightly slower scale-out but zero errors in prod.
```

⚠ Gotchas

Gotcha	Detail
Health check type	Default is EC2 health check (just "is the VM running"). Set to ELB to check actual app health
Termination policy	Default terminates oldest launch config first, then closest to billing hour
AZ rebalancing	ASG auto-balances across AZs even without scaling events — can terminate instances unexpectedly
Scale-in protection	Individual instances can be protected from scale-in — useful for "leader" instances
Desired < Min	You can't set Desired below Min. Common Terraform bug
Cooldown vs warmup	Cooldown = pause after scaling. Warmup = time new instance counts toward metrics

---

2.4 Launch Templates vs Launch Configurations

🟢 What It Is in Simple Terms

Both define "what kind of EC2 instance to launch" — the blueprint for your ASG. Launch Templates are the modern replacement. Launch Configurations are legacy and AWS has stopped adding features to them.

⚙️ Key Differences

Feature	Launch Config	Launch Template
Versioning	❌ None	✅ Yes (v1, v2, v3)
Spot + On-Demand mix	❌ No	✅ Yes
Multiple instance types	❌ No	✅ Yes
T2/T3 Unlimited	❌ No	✅ Yes
Dedicated Hosts	❌ No	✅ Yes
Capacity Reservations	❌ No	✅ Yes
AWS recommendation	⚠ Legacy	✅ Use this

AWS CLI — Create a Launch Template

```
aws ec2 create-launch-template \
  --launch-template-name my-web-lt \
  --version-description "v1 - initial" \
  --launch-template-data '{
    "ImageId": "ami-0abcdef1234567890",
    "InstanceType": "m6i.large",
    "KeyName": "my-keypair",
    "SecurityGroupIds": ["sg-12345678"],
    "IamInstanceProfile": {
      "Arn": "arn:aws:iam::123456789:instance-profile/my-role"
    },
    "UserData": "IyEvYmluL2Jhc2gKeXVtIHVwZGF0ZSAteQ==",
    "MetadataOptions": {
      "HttpTokens": "required",
      "HttpEndpoint": "enabled"
    },
    "TagSpecifications": [{
      "ResourceType": "instance",
      "Tags": [{"Key": "Environment", "Value": "prod"}]
    }]
  }'
```

> ⚠ **Key Interview Point:** Launch Templates support **mixed instance policies** — you can specify a base instance type + overrides, letting ASG use m6i.large OR m6a.large OR m5.large, mixing Spot

> and On-Demand in one ASG. This is critical for Spot diversification.

💬 Short Interview Answer

"Launch Templates are the modern, versioned replacement for Launch Configurations. The critical advantage is versioning — you can have v1, v2, v3 and roll back instantly. They also support mixed instance policies, letting a single ASG run multiple instance types and mix Spot with On-Demand. Launch Configurations are legacy — AWS hasn't added features to them in years. Always use Launch Templates."

2.5 Elastic Load Balancing — ALB, NLB, CLB, GWLB

🟢 What It Is in Simple Terms

A load balancer sits in front of your servers and distributes incoming traffic across them. It also health-checks your instances and stops sending traffic to unhealthy ones. AWS has 4 types — each operating at different layers of the network stack for fundamentally different use cases.

🔍 Why Load Balancers Exist / What Problem They Solve

```
Without a load balancer:
├── Single EC2 = single point of failure
├── Scale out = users must know each IP address
├── Unhealthy instance = requests fail until manual removal
└── TLS certificates = managed on each instance individually

With a load balancer:
├── Single DNS endpoint regardless of fleet size
├── Automatic health check → unhealthy instances removed from rotation
├── TLS terminated centrally (ACM certificate on LB — free, auto-renewed)
├── Cross-AZ distribution → AZ failure doesn't take down the service
└── Sticky sessions, content routing, authentication offload at the edge
```

⚙️ How It Works Internally

```
Internet → Load Balancer → Target Group → EC2/ECS/Lambda/IP

                    ┌──────────────────────────────────────┐
                    │           Load Balancer              │
                    │   (Multi-AZ, fully managed by AWS)   │
                    │   Listener:443 → Listener Rules       │
                    │   → Target Group selection           │
                    └──────────────────┬
```

```
                                  │
                     ┌────────────┴────────────┐
                     │                         │
              ┌──────▼──────┐           ┌──────▼──────┐
              │ Target Group A │         │ Target Group B │
              │ /api/users/*   │         │ /api/orders/*  │
              │ Health: /health│         │ Health: /ping  │
              └──────┬──────┘           └──────┬──────┘
                     │      │                   │      │
            ┌────────▼─┐  ┌─▼────────┐     ┌────▼─┐  ┌─▼────┐
            │ EC2      │  │ EC2      │     │ EC2  │  │ EC2  │
            │ AZ-a     │  │ AZ-b     │     │ AZ-a │  │ AZ-b │
            └──────────┘  └──────────┘     └──────┘  └──────┘

Key internal components:
├── Listener:       port + protocol the LB accepts traffic on
├── Listener rules: ordered conditions → target group mapping
├── Target group:   logical group of targets with shared health checks
└── Target:         EC2 instance, IP address, Lambda, or ALB
```

🧩 The 4 Load Balancer Types

```
┌──────────────┬────────────────┬──────────────────────────────────┬──────────────────
────────┐
│ Type         │ OSI Layer      │ Primary Use Case                 │ Key
Capability       │
├──────────────┼────────────────┼──────────────────────────────────┼──────────────────
────────┤
│ ALB          │ Layer 7        │ Web apps, microservices, APIs    │ Content-
based routing│
│ NLB          │ Layer 4        │ TCP/UDP, PrivateLink, low lat    │ Static IP,
source IP │
│ CLB          │ Layer 4 + 7    │ Legacy (do not use)              │ Deprecated
│
│ GWLB         │ Layer 3        │ Security appliances (firewall)   │ GENEVE,
bump-in-wire │
└──────────────┴────────────────┴──────────────────────────────────┴──────────────────
────────┘
```

🧩 ALB — Application Load Balancer (Layer 7)

What Makes ALB Different

```
ALB terminates the HTTP connection at the load balancer.
It reads the full HTTP request — method, headers, path, query string —
before deciding where to route it.
```

```
This is what enables content-based routing:
the LB actually understands the request, not just the TCP connection.
```

ALB Request Flow (Internals)

```
HTTPS request — step by step:

1. Client → ALB node (port 443, TLS)
2. ALB decrypts TLS using ACM certificate (TLS termination)
3. ALB reads HTTP request:
   - Method:  GET
   - Host:    api.company.com
   - Path:    /api/users/123
   - Headers: Authorization: Bearer xyz, X-App-Version: 2
4. ALB evaluates listener rules in PRIORITY ORDER (1 = highest):
   Rule 1:  Path is /api/admin/* AND Source IP is 10.0.0.0/8 → Admin TG
   Rule 2:  Host is api.company.com AND Path is /api/* → API TG
   Rule 3:  Path is /static/* → S3 redirect (301)
   Default: Forward to web-default TG
5. ALB selects target using routing algorithm:
   Default: round-robin
   Optional: least-outstanding-requests (better for variable-length
responses)
6. ALB opens connection to selected EC2 (on port 80 or 443)
7. ALB forwards request — adds X-Forwarded-For, X-Forwarded-Proto headers
8. Response flows back through ALB to client

⚠ EC2 sees ALB's PRIVATE IP as source — NOT the client IP.
   Use X-Forwarded-For header to get the real client IP.
   Configure PROXY protocol on NLB for real IP preservation.
```

ALB Routing Rules — Full Reference

```
Condition types (can combine multiple with AND):
├── host-header:      api.company.com (supports wildcards: *.company.com)
├── path-pattern:     /api/users/* (supports * and ?)
├── http-header:      X-Custom-Header: mobile
├── query-string:     version=2 or ?platform=ios
├── http-request-method: GET, POST, DELETE, etc.
├── source-ip:        CIDR block (10.0.0.0/8, 203.0.113.0/24)
└── ip-based (combined): source IP range

Action types (what to do when conditions match):
├── forward:          route to one or more target groups (weighted split!)
├── redirect:         HTTP 301/302 to new URL (path rewrite, protocol
change)
├── fixed-response:   return static HTTP response (200 OK, 503
Maintenance)
├── authenticate-cognito: require Cognito authentication before forwarding
```

```
└── authenticate-oidc: require OIDC provider (Okta, Google, etc.) before
forwarding

Weighted target groups (for canary deployments):
Rule action: forward to:
  - Target Group v1: weight 90
  - Target Group v2: weight 10
→ ALB sends 10% of traffic to new version — canary deployment!
   No Route 53, no separate DNS — done at ALB rule level.
```

```
# Create ALB listener rule — path-based routing to different ECS services
aws elbv2 create-rule \
  --listener-arn arn:aws:elasticloadbalancing:...:listener/app/prod-
alb/... \
  --priority 10 \
  --conditions '[
    {"Field":"path-pattern","PathPatternConfig":{"Values":
["/api/payments/*"]}},
    {"Field":"http-header","HttpHeaderConfig":{"HttpHeaderName":"X-API-
Version","Values":["v2"]}}
  ]' \
  --actions '[
    {"Type":"forward","TargetGroupArn":"arn:...payments-v2-tg"}
  ]'

# Weighted forward — canary deployment at ALB level
aws elbv2 create-rule \
  --listener-arn arn:...:listener/... \
  --priority 20 \
  --conditions '[{"Field":"path-pattern","PathPatternConfig":{"Values":
["/api/orders/*"]}}]' \
  --actions '[{
    "Type": "forward",
    "ForwardConfig": {
      "TargetGroups": [
        {"TargetGroupArn": "arn:...orders-v1-tg", "Weight": 90},
        {"TargetGroupArn": "arn:...orders-v2-tg", "Weight": 10}
      ],
      "StickinessConfig": {"Enabled": true, "DurationSeconds": 300}
    }
  }]'
# 10% traffic to v2 — weighted canary at ALB layer, no DNS change needed
```

ALB Target Types

```
Target types — what ALB can route to:
├── instance:   EC2 instance ID (port configured on target group)
├── ip:         Any IP address in your VPC, VPN, or Direct Connect
│               → on-prem servers as ALB targets!
```

```
│                → ECS tasks with awsvpc networking (task IP, not host IP)
└── lambda:      Single Lambda function
                 → ALB invokes Lambda with event payload
                 → Lambda returns JSON response converted to HTTP

Target group stickiness (sticky sessions):
├── Duration-based: AWSALB cookie (LB-generated, set TTL)
├── Application-based: AWSALBAPP cookie (app sets cookie, LB reads it)
└── Disabled: pure round-robin or least-outstanding-requests

When stickiness breaks:
├── Instance terminates or becomes unhealthy → user re-distributed
├── Cookie expires → user re-distributed
└── Session replication needed for stateful apps across re-distribution
```

ALB Security Integration

```
WAF attachment:
aws wafv2 associate-web-acl \
  --web-acl-arn arn:aws:wafv2:...:regional/webacl/prod-waf/... \
  --resource-arn arn:aws:elasticloadbalancing:...:loadbalancer/app/prod-
alb/...
# WAF evaluates EVERY request before ALB rules
# Block SQL injection, XSS, rate limiting, geo-blocking at ALB edge

ALB Access Logs → S3 (critical for debugging):
aws elbv2 modify-load-balancer-attributes \
  --load-balancer-arn arn:...prod-alb... \
  --attributes \
    Key=access_logs.s3.enabled,Value=true \
    Key=access_logs.s3.bucket,Value=my-alb-logs \
    Key=access_logs.s3.prefix,Value=prod-alb \
    Key=idle_timeout.timeout_seconds,Value=120

ALB access log fields (each line = one request):
timestamp, elb, client:port, target:port, request_processing_time,
target_processing_time, response_processing_time, elb_status_code,
target_status_code, received_bytes, sent_bytes, "request",
"user_agent", ssl_cipher, ssl_protocol, target_group_arn, "trace_id",
"domain_name", "chosen_cert_arn", matched_rule_priority, ...

# Query access logs with Athena:
SELECT
  target_status_code,
  COUNT(*) AS count,
  AVG(target_processing_time) AS avg_ms
FROM alb_logs
WHERE elb_status_code = '502'
  AND year='2024' AND month='01' AND day='15'
GROUP BY target_status_code
ORDER BY count DESC;
```

🧩 NLB — Network Load Balancer (Layer 4)

What Makes NLB Different

```
NLB does NOT terminate the TCP connection by default.
It operates at the transport layer — it sees TCP/UDP packets, not HTTP.
It routes based on: destination IP + port + protocol.

NLB forwards the TCP connection to a target.
The TCP handshake happens between CLIENT and EC2 (NLB is transparent).
This is why NLB preserves the source IP — it's not breaking the
connection.

Performance characteristics:
├── Sub-millisecond latency (vs ALB ~1-10ms additional)
├── Handles millions of requests per second per AZ
├── No content inspection overhead
└── Connection-level routing (not request-level)
```

NLB Key Features

```
1. Static Elastic IPs per AZ:
   ├── Each AZ gets ONE fixed IP (assignable EIP)
   ├── Customers can whitelist these IPs in their firewall
   └── ALB has NO static IPs — DNS name resolves to changing IPs

2. Source IP preservation:
   ├── EC2 receives packets with CLIENT'S real IP as source
   ├── EC2 security group must allow traffic from: 0.0.0.0/0 or client
CIDRs
   │   (NOT from NLB IP — NLB doesn't have a SG to reference)
   └── Contrast with ALB: EC2 SG allows ALB's security group

3. TLS termination (optional):
   ├── NLB CAN terminate TLS (like ALB) using ACM cert
   ├── Or: pass-through TCP (client TLS to EC2 directly)
   └── Use pass-through when: E2E encryption required, mutual TLS (mTLS)

4. PrivateLink requirement:
   ├── AWS PrivateLink REQUIRES NLB as the service endpoint
   ├── NLB fronts your service → expose as PrivateLink endpoint
   └── Other accounts connect via VPC Interface Endpoint → NLB → your
service
```

```
# NLB with TCP listener + Elastic IP per AZ
aws elbv2 create-load-balancer \
  --name prod-payments-nlb \
  --type network \
  --scheme internet-facing \
  --subnet-mappings \
    SubnetId=subnet-az1,AllocationId=eipalloc-1 \
    SubnetId=subnet-az2,AllocationId=eipalloc-2 \
    SubnetId=subnet-az3,AllocationId=eipalloc-3

# Create TLS listener (NLB terminates TLS)
aws elbv2 create-listener \
  --load-balancer-arn arn:...prod-payments-nlb... \
  --protocol TLS \
  --port 443 \
  --certificates CertificateArn=arn:aws:acm:...:certificate/... \
  --default-actions Type=forward,TargetGroupArn=arn:...payments-tg

# Create TCP pass-through listener (E2E TLS — NLB doesn't decrypt)
aws elbv2 create-listener \
  --load-balancer-arn arn:...prod-nlb... \
  --protocol TCP \
  --port 443 \
  --default-actions Type=forward,TargetGroupArn=arn:...tg
# Client TLS handshake goes all the way to EC2 — NLB is transparent
```

NLB Health Checks

```
NLB health check protocols:
├── TCP:    opens a connection — succeeds if TCP handshake completes
├── HTTP:   GET /path — succeeds on specific status codes
└── HTTPS:  GET /path with TLS — succeeds on specific status codes

NLB TCP health check behavior:
├── NLB opens TCP connection to target
├── Immediately closes it (RST packet)
└── Target may log "connection reset" errors — this is NORMAL

NLB health check is separate from client traffic:
├── Health check ALWAYS uses NLB's IP (different from client IP)
└── Security group on EC2 must allow NLB health check source:
    For instance targets: allow NLB IP ranges (VPC CIDR)
    For IP targets:       allow NLB's subnet CIDRs
```

🧩 CLB — Classic Load Balancer (Legacy)

```
Operates at: Layer 4 + Layer 7 (limited)
Status:      LEGACY — AWS has not added features since 2016

Why it's inferior to ALB + NLB:
├── No target groups (routes to instances directly)
├── No path-based routing (all traffic to same backend)
├── No SNI support (one SSL cert per CLB)
├── No WebSocket support
├── No Lambda targets
└── No WAF integration

Interview answer:
"CLB is the original generation load balancer. It doesn't support
path-based routing, target groups, or most modern features.
All new workloads use ALB (Layer 7) or NLB (Layer 4). I've never
used CLB in any system built after ~2016."
```

🧩 GWLB — Gateway Load Balancer (Layer 3)

What GWLB Does

```
GWLB solves: "How do I route ALL traffic through a security appliance
             (firewall, IDS/IPS, DPI) without changing my app?"

Traffic flow — bump-in-wire pattern:
Internet → IGW → GWLB Endpoint → GWLB → Firewall Fleet → GWLB → App


┌──────────────────────────────────────────────────────────────────┐
│ Without GWLB:                                                    │
│ Internet → IGW → ALB → EC2 App                                   │
│ (no inspection — traffic goes direct)                            │
│                                                                  │
│ With GWLB:                                                       │
│ Internet → IGW → GWLB Endpoint (VPC route table magic)           │
│         → GWLB → Palo Alto / CheckPoint fleet (inspect/filter)   │
│         → GWLB → ALB → EC2 App                                   │
└──────────────────────────────────────────────────────────────────┘


Key technical details:
├── Uses GENEVE protocol (port 6081) — encapsulates original packets
├── Firewall appliance sees original packet intact (source IP preserved)
├── GWLB distributes to firewall fleet (5-tuple hash for flow stickiness)
├── Flow stickiness: same client flow always goes to same firewall
instance
│   (critical: stateful firewalls must see all packets of a flow)
└── Deployed as VPC Endpoint Service (PrivateLink-like architecture)
```

```
# Create GWLB for firewall appliance fleet
aws elbv2 create-load-balancer \
  --name prod-security-gwlb \
  --type gateway \
  --subnets subnet-az1 subnet-az2

# Create target group for firewall appliances
aws elbv2 create-target-group \
  --name firewall-fleet \
  --protocol GENEVE \
  --port 6081 \
  --target-type instance \
  --vpc-id vpc-123

# Create GWLB Endpoint Service
aws ec2 create-vpc-endpoint-service-configuration \
  --gateway-load-balancer-arns arn:...prod-security-gwlb...

# In the consumer VPC — route all traffic through GWLB endpoint
# VPC route table: 0.0.0.0/0 → GWLB Endpoint (instead of IGW directly)
```

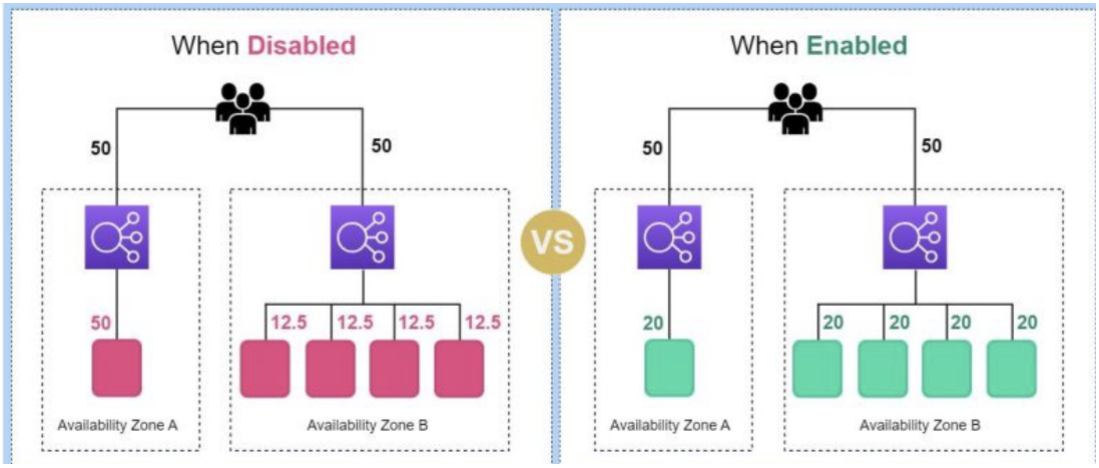
🧩 Target Groups — Deep Dive

```
Target Group = logical pool of targets + health check configuration

Registration methods:
├── Manual: explicitly register instance IDs or IPs
├── ASG: Auto Scaling Group registers instances automatically on launch
└── ECS: service automatically registers/deregisters task IPs

Health check configuration (critical to tune correctly):
├── Protocol:           HTTP, HTTPS, TCP, TLS, HTTP2, GRPC
├── Path:               /health (should be lightweight — no DB queries!)
├── Healthy threshold:  2 consecutive successes → mark healthy
├── Unhealthy threshold: 5 consecutive failures → mark unhealthy
├── Timeout:            5 seconds (must be < interval)
├── Interval:           30 seconds (time between checks)
└── Success codes:      200, 200-299, or specific codes

Health check tuning for production:
Fast detection:  interval=10s, threshold=2, timeout=5s
                 → unhealthy in 20 seconds (2 × 10s)
Slow detection:  interval=30s, threshold=5, timeout=29s
                 → unhealthy in 150 seconds (5 × 30s)

⚠ Health check endpoint MUST be stateless and fast.
   Never: SELECT 1 FROM huge_table (slow → timeout → false positive)
   Always: return 200 if app can process requests (check DB connectivity
briefly)
```

```
Deregistration delay (connection draining):
├── Default: 300 seconds
├── During deregistration: LB stops sending NEW requests
│   In-flight requests continue until: completion OR timeout
└── Tune to: max request processing time × safety margin
   For fast APIs (< 1s): set to 30 seconds
   For long-running tasks (30s): set to 60-90 seconds
   For slow deployments: CodeDeploy waits for drain to complete
```

```
# Target group with tuned health check
aws elbv2 create-target-group \
  --name prod-api-tg \
  --protocol HTTP \
  --port 8080 \
  --vpc-id vpc-123 \
  --health-check-protocol HTTP \
  --health-check-path /health \
  --health-check-interval-seconds 10 \
  --health-check-timeout-seconds 5 \
  --healthy-threshold-count 2 \
  --unhealthy-threshold-count 3 \
  --target-type ip \
  --load-balancing-algorithm-type least_outstanding_requests

# Register specific IP targets (ECS awsvpc networking)
aws elbv2 register-targets \
  --target-group-arn arn:...prod-api-tg \
  --targets Id=10.0.1.15,Port=8080 Id=10.0.2.33,Port=8080
```

🧩 Cross-Zone Load Balancing




```
Cross-zone load balancing = distribute requests evenly across ALL
                            targets in ALL AZs, not just the LB's AZ
```

```
Without cross-zone (default NLB):

┌──────────────────────────────────────────┐
│  AZ-a LB node          AZ-b LB node      │
│  receives 50%          receives 50%      │
│                                          │
│  Targets: 2 in AZ-a   Targets: 8 in AZ-b│
│  Each gets 25%         Each gets 6.25%   │
│  (AZ-a targets overloaded!)              │
└──────────────────────────────────────────┘

With cross-zone enabled:

┌──────────────────────────────────────────┐
│  AZ-a LB node: forwards to ANY of 10    │
│  AZ-b LB node: forwards to ANY of 10    │
│  Each target gets exactly 10% of traffic│
└──────────────────────────────────────────┘

Cross-zone defaults:
├── ALB: ENABLED by default — no inter-AZ charge
├── NLB: DISABLED by default — if enabled: $0.01/GB inter-AZ charge
└── GWLB: DISABLED by default — if enabled: $0.01/GB inter-AZ charge

Production guidance:
├── ALB: leave on (default) — uneven instances per AZ common with ASGs
└── NLB: weigh cost vs uniformity need
    If AZ instance counts are always equal → no benefit, leave off
    If AZ counts vary → enable to prevent hot spots
```

🧩 ALB vs NLB — Security Group Behavior

```
This is one of the most commonly misunderstood differences.

ALB Security Group model:
Internet → ALB (has its own SG: sg-alb)
         → EC2 (SG rule: allow from sg-alb on port 8080)

ALB has a security group.
EC2 references ALB's SG in its inbound rule.
EC2 NEVER sees the client IP — it sees the ALB private IP.
Get client IP from: X-Forwarded-For HTTP header.

NLB Security Group model:
Internet → NLB (NO security group — NLB is transparent)
         → EC2 (SG rule: allow 0.0.0.0/0 port 443 OR allow specific CIDRs)

NLB has NO security group.
EC2 sees the CLIENT'S real IP as source.
EC2 SG must allow the CLIENT CIDRs directly.

⚠ Common mistake:
```

```
    Using NLB but adding SG rule "allow from NLB SG" on EC2.
    This DOESN'T WORK — NLB has no SG.
    You must allow the actual client source IP ranges.

Exception: When NLB targets EC2 instances (not IPs), you can enable
           "client IP preservation" to be disabled per target group,
           and NLB's internal IPs become the source.
           This is off by default for instance targets, on for IP targets.
```

🏭 Real World Production Architectures

Architecture 1: Microservices API Gateway

```
            ┌──────────────────────────────────────────────────────┐
            │                  Single ALB — one DNS name           │
            │              (api.company.com → ALB)                 │
            └──────────────────────────────────────────────────────┘
                │ /api/users/*      │ /api/payments/*  │ /api/orders/*
                ▼                   ▼                  ▼

        ┌──────────┐        ┌──────────────┐     ┌──────────────┐
        │ Users TG │        │ Payments TG  │     │  Orders TG   │
        │ ECS Fargate│      │ ECS Fargate  │     │  ECS Fargate │
        │ 3 tasks  │        │ 5 tasks      │     │  4 tasks     │
        └──────────┘        └──────────────┘     └──────────────┘

       (scales 1-20)       (scales 2-50)       (scales 1-30)

Benefits:
├── One ALB, one ACM cert, one DNS name
├── Each service auto-scales independently by queue depth / CPU
├── Different health check paths per service
└── WAF attached once at ALB — covers all services
```

Architecture 2: PrivateLink Service Provider

```
Service provider account:          Consumer accounts (100+ customers)

┌──────────────────────────┐        ┌──────────────────────┐
│  Your service            │        │  Customer VPC        │
│  NLB (required!)         │        │  Interface Endpoint  │
│  ↑                       │◄───────│  (PrivateLink)       │
│  ECS tasks / EC2         │        │                      │
└──────────────────────────┘        │  App calls           │
                                    │  vpce-svc-xxx.us-east│
                                    └──────────────────────┘

PrivateLink requires NLB — not ALB, not GWLB.
Customer traffic never traverses public internet.
Your source IP appears as NLB's private IP to your app.
```

Architecture 3: Blue/Green Deployment via ALB Weighted Rules

```
Deployment pipeline:
Step 1: Deploy v2 to Green TG (no traffic yet)
        Blue TG:  100 weight
        Green TG: 0 weight

Step 2: Canary (5% to green)
        Blue TG:  95 weight
        Green TG: 5 weight
        → Monitor CloudWatch: error rate, latency
        → If alarm: set Green to 0 immediately (no DNS change needed)

Step 3: Expand canary (25%)
        Blue TG:  75 weight
        Green TG: 25 weight

Step 4: Full cutover
        Blue TG:  0 weight
        Green TG: 100 weight

Step 5: Old blue instances deregister after drain period
        → Terminate old EC2 / scale down blue ASG

All done at ALB rule level — no Route 53 TTL delay.
Rollback = change weights back instantly.
```

Architecture 4: Hybrid On-Premises + Cloud

```
On-premises data center         AWS VPC
┌──────────────────────┐        ┌──────────────────────────────────┐
│  Legacy app server   │        │  ALB (internet-facing)           │
│  IP: 10.200.1.50     │        │                                  │
│  port 8080           │        │  Target Group:                   │
│                      │◄─Direct─│  ├── EC2 i-001 (10.0.1.10:8080) │
│                      │ Connect │  ├── EC2 i-002 (10.0.2.15:8080) │
│                      │        │  └── 10.200.1.50:8080 ← ON-PREM │
└──────────────────────┘        │      (IP target type)            │
                                └──────────────────────────────────┘

IP target type supports:
├── Any IP reachable from VPC (via VPN, Direct Connect, peering)
└── Gradual migration: on-prem IPs in TG alongside EC2 IPs
    Shift weight from on-prem → cloud during migration
```

🩺 Operational Troubleshooting

HTTP 5xx Error Diagnosis

```
502 Bad Gateway:
├── Target returned invalid HTTP response (malformed)
├── Target connection refused (app not running on target port)
├── Target closed connection before response sent
└── TLS certificate error when ALB talks to target over HTTPS
Diagnosis: check ALB access logs → target_status_code field

503 Service Unavailable:
├── ALL targets in target group are unhealthy
│   → check health check endpoint: is /health responding correctly?
├── No targets registered in target group (ASG not attached)
└── Target group capacity = 0 (scaled to zero, Lambda concurrency 0)
Diagnosis: aws elbv2 describe-target-health --target-group-arn ...

504 Gateway Timeout:
├── Target is reachable but NOT responding within idle timeout
├── Target is processing request longer than ALB
idle_timeout.timeout_seconds
└── Typically: long-running queries, large file uploads, slow downstream
Fix: increase idle_timeout (default 60s → 300s for slow operations)
     OR optimize target response time

ELB HTTP 400 (client-side bad request):
└── Malformed HTTP/1.1 request headers
    Missing Host header, invalid characters, H2 protocol mismatch

Target group shows all instances as unhealthy:
├── Health check path doesn't exist (404 on /health)
├── App not started yet (grace period issue)
├── SG blocks health check traffic (check source for ALB)
└── App returns 200 but health check expects 200-299 only (check matcher)
```

```
# Check target health states
aws elbv2 describe-target-health \
  --target-group-arn arn:...prod-api-tg

# Output shows per-target:
# State: healthy | unhealthy | initial | draining | unused
# Reason: Target.ResponseCodeMismatch | Target.Timeout | Target.NotInUse
# Description: "Health checks failed with these codes: [404]"

# Check ALB attributes — common tuning issues
aws elbv2 describe-load-balancer-attributes \
  --load-balancer-arn arn:...prod-alb

# Key attributes:
# idle_timeout.timeout_seconds: 60 (default) — increase for slow backends
# routing.http2.enabled: true (default)
# access_logs.s3.enabled: false (default) — always enable in production!
# deletion_protection.enabled: false — enable for prod ALBs!
```

```
# Check listener rules (debug routing issues)
aws elbv2 describe-rules \
  --listener-arn arn:...listener/...
# Rules are evaluated in priority order (1 = first)
# Default rule always has priority "default" and no conditions
```

CloudWatch Metrics for ELB Operations

```
ALB key CloudWatch metrics:
├── RequestCount:              total requests per period
├── TargetResponseTime:        backend latency (P50, P95, P99!)
├── HTTPCode_Target_5XX_Count: backend errors (app bugs)
├── HTTPCode_ELB_5XX_Count:    LB errors (502/503/504 — infra issues)
├── HTTPCode_ELB_4XX_Count:    client errors (bad requests)
├── HealthyHostCount:          targets currently healthy
├── UnHealthyHostCount:        targets currently failing health checks
└── ActiveConnectionCount:     open TCP connections to ALB

NLB key CloudWatch metrics:
├── ProcessedBytes:          bytes processed (costs $$ for cross-zone)
├── TCP_Target_Reset_Count:  unexpected RST from targets (app crashes)
├── TCP_Client_Reset_Count:  clients aborting connections
├── HealthyHostCount:        healthy targets
└── ActiveFlowCount:         concurrent active TCP flows

Alarms to always configure:
├── HealthyHostCount < 1 → PagerDuty/SNS (CRITICAL — no healthy targets)
├── UnHealthyHostCount > 0 for 5 min → Warning
├── TargetResponseTime P99 > 2s → Latency SLO breach
└── HTTPCode_ELB_5XX_Count > 10/min → Infrastructure problem
```

💬 Common Interview Questions and Strong Answers

Q: "What's the difference between ALB and NLB? When do you choose each?"

"ALB operates at Layer 7 — it understands HTTP and makes routing decisions based on request content: path, host header, HTTP method, query parameters. It's the right choice for web applications and microservices where you need content-based routing, WebSocket, or gRPC. It terminates TLS centrally and replaces source IP with its own.

NLB operates at Layer 4 — TCP/UDP with no content inspection. It has three key advantages ALB doesn't: one static Elastic IP per AZ (useful for customer IP whitelisting), source IP preservation (backend sees real client IP), and it's required for PrivateLink. It handles millions of connections per second with sub-millisecond latency, making it the choice for financial systems, gaming, or any protocol-level workload.

In practice I often stack them: NLB for the fixed IP and PrivateLink capability, then ALB behind it for application-layer routing."

Q: "Your ALB is returning 502s for about 5% of requests. How do you diagnose this?"

"502 means the ALB reached a target but got an invalid response. First I'd check ALB access logs in S3 and filter for 502s — the target_status_code field tells me what the backend actually returned. If it shows '-', the connection was refused or reset before a response. If it shows a status code, it means the app returned a malformed HTTP response.

I'd also look at HealthyHostCount — if some instances are unhealthy, 5% failing makes sense. Then I'd SSH into an affected instance and check app logs around the 502 timestamps. Common culprits: app crashing under load, connection pool exhaustion causing refused connections, or a memory leak causing slow responses that ALB times out.

Finally I'd check idle_timeout — if our ALB is at default 60 seconds and we have requests taking longer than that, ALB closes the connection and returns 504, but the log shows 502 in some versions."

Q: "How would you do a zero-downtime deployment with ALB?"

"My preferred approach is ALB weighted target groups — no Route 53 TTL delays. I deploy v2 to a new target group, start it at weight 0. Then I gradually shift: 5% to v2, watch CloudWatch for error rate and latency alarms. If clean after 10 minutes, go to 25%, then 50%, then 100%.

Rollback is instant: set v2 weight to 0. The whole process happens at ALB rule level — no DNS changes, no waiting for TTL propagation.

For the database layer, I use the expand-contract pattern: v2 must be backward-compatible with the schema while v1 is still running. Once v1 is completely gone from the target group, I can clean up old columns.

CodeDeploy can automate this entire flow with automatic rollback based on CloudWatch alarms — if error rate exceeds threshold during the canary phase, it reverses automatically."

Q: "Why does enabling cross-zone load balancing on NLB cost money but not on ALB?"

"When cross-zone is enabled, traffic from one AZ's LB node is forwarded to targets in other AZs. AWS charges inter-AZ data transfer at $0.01/GB.

ALB has cross-zone on by default and absorbs that cost into the ALB pricing — AWS made the decision that uniform distribution matters enough to include it.

NLB keeps it off by default and charges explicitly because NLB is frequently used for very high-throughput workloads where that inter-AZ traffic cost could be significant — think hundreds of TB per month. If you have equal instance counts across AZs, cross-zone adds no value and only adds cost, so NLB lets you choose."

Q: "Explain how ALB authenticates users without application code changes."

"ALB has native OIDC and Cognito authentication built into listener rules. You configure the authenticate-cognito or authenticate-oidc action before the forward action. When a user hits your ALB, if they don't have

a valid session cookie, ALB redirects them to the identity provider — Cognito, Okta, Google, etc. After authentication, the IdP redirects back to ALB with a code, ALB exchanges it for tokens, validates them, and then forwards the request to your backend with added headers: X-Amzn-Oidc-Identity, X-Amzn-Oidc-Accesstoken, X-Amzn-Oidc-Data.

Your application receives an already-authenticated request with user identity in the headers — zero auth code in the app itself. This is great for internal tools, admin dashboards, or any app where you want to offload auth entirely to the infrastructure layer."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
ALB source IP → use X-Forwarded-For	EC2 sees ALB's private IP. Never block by "source IP" on EC2 SG when behind ALB — use XFF header
NLB has no security group	Cannot reference "NLB SG" in EC2 security group rules. Must allow client CIDR ranges directly
Cross-zone NLB costs money	Enabling cross-zone on NLB adds $0.01/GB inter-AZ data transfer charge
ALB idle timeout = 504 risk	Long-running requests (file upload, slow query) need idle_timeout increased from default 60s
Deregistration delay = slow deploys	Default 300s drain means CodeDeploy waits 5 minutes per batch. Reduce to 30s for fast API deploys
Health check must be fast and stateless	Health check that does DB queries causes false positives under DB load → instances incorrectly removed
NLB TCP health check = RST noise	NLB opens and immediately closes TCP for health checks → connection reset in app logs — this is normal
ALB rule limit	100 rules per listener (soft limit). Plan routing architecture for microservices with many routes
GWLB flow stickiness = required	Stateful firewalls must see all packets of a flow. Disable cross-protocol LB to ensure stickiness
ALB deletion protection	Default OFF — a terraform destroy or accident deletes production ALB. Enable deletion_protection.enabled
NLB static IP changes on recreation	If you delete and recreate an NLB, the Elastic IP re-assignments must be manual. Use Terraform to manage
Lambda targets have payload limit	ALB → Lambda max request body: 1MB. Larger bodies need S3 presigned URL pattern instead

---

🔗 Integration With Other AWS Services

```
ECS / Fargate:
├── ALB: awsvpc networking → task IP registered as IP target
│   ECS service creates/destroys tasks → auto-registers/deregisters from
TG
├── NLB: PrivateLink services — expose Fargate service via NLB
└── Health check: ECS marks task unhealthy if LB health check fails →
replaces task

Auto Scaling Groups:
├── ASG → Target Group attachment → instances auto-register on launch
├── Scale-in: ASG deregisters instance → LB drains → then terminates
└── Health checks: ALB/NLB health check can replace ASG health check
    (more meaningful than EC2 status check — app-level health)

CodeDeploy:
├── ALB: weighted target groups → blue/green or canary deployments
├── NLB: also supported for blue/green
└── Automatic rollback: CodeDeploy monitors LB metrics during deployment

ACM (Certificate Manager):
├── Free SSL certs on ALB/NLB — auto-renewed by AWS
├── SNI on ALB: multiple certs, multiple domains on one ALB
└── Certificate rotation: ACM renews → ALB automatically picks up new cert

WAF v2:
├── Only ALB (not NLB, not GWLB) supports direct WAF attachment
└── WAF rules evaluated before any ALB listener rules

Route 53:
├── Alias records: Route 53 → ALB DNS name (no IP needed)
├── Health checks: Route 53 health check → ALB DNS → marks record
unhealthy
└── Failover: Route 53 + health check → failover between regions

CloudWatch:
├── All LB metrics flow to CloudWatch automatically
├── Access logs → S3 → Athena for deep analysis
└── Alarm on HealthyHostCount < 1 → SNS → PagerDuty

Global Accelerator:
├── Anycast IPs → nearest AWS edge → then AWS backbone → ALB/NLB
├── Reduces latency for global users (vs direct internet routing to ALB)
└── Provides static IPs for ALB (ALB has no static IPs natively)
```

📌 Quick Reference Cheat Sheet

Feature	ALB	NLB	GWLB
OSI Layer	7 (HTTP)	4 (TCP/UDP)	3 (IP)

Feature	ALB	NLB	GWLB
Content routing	✅ Path/host/header/query	❌ Port only	❌ None
Static IP	❌ (use Global Accelerator)	✅ 1 EIP per AZ	❌
Source IP preserved	❌ (X-Forwarded-For)	✅ Native	✅
Security group	✅ Required	❌ None	❌ None
WebSocket	✅	✅ (TCP passthrough)	N/A
gRPC	✅	✅ (TCP passthrough)	N/A
PrivateLink	❌	✅ Required	❌
WAF support	✅	❌	❌
Auth offload	✅ Cognito/OIDC	❌	❌
Cross-zone default	✅ On (free)	❌ Off (costs $$)	❌ Off
TLS termination	✅	✅ optional	❌
Lambda targets	✅	❌	❌
Protocols	HTTP, HTTPS	TCP, UDP, TLS, TCP_UDP	IP (all)
Use case	Web/API/microservices	Low-latency, PrivateLink	Security appliances

---

2.6 EC2 Placement Groups

🟢 What It Is in Simple Terms

Placement Groups tell AWS HOW to physically place your EC2 instances relative to each other. Sometimes you want them on the same physical hardware for speed. Sometimes you want them as far apart as possible for resilience.

🧩 Three Types

1. Cluster Placement Group

```
Goal: Maximum network performance between instances

Physical layout:
┌──────────────────────────────────┐
│     Single Rack, Single AZ       │
│  ┌─────┐  ┌─────┐  ┌─────┐      │
│  │ EC2 │  │ EC2 │  │ EC2 │      │
│  └─────┘  └─────┘  └─────┘      │
```

```
└──────────────────────────────────────┘

Network: Up to 100 Gbps between instances
Latency: Single-digit microseconds

Use cases:
├── HPC (tightly coupled MPI jobs)
├── Big data analytics (Spark/Hadoop with huge shuffles)
└── Machine learning training (GPU clusters)

⚠ Risk: If rack fails, ALL instances fail together
⚠ Risk: May get "insufficient capacity" if you try to add
          instances later — launch all instances at once
```

---

2. Spread Placement Group

```
Goal: Maximum isolation / resilience

Physical layout:
┌──────┐   ┌──────┐   ┌──────┐
│Rack 1│   │Rack 2│   │Rack 3│
│ EC2  │   │ EC2  │   │ EC2  │
└──────┘   └──────┘   └──────┘
(Different racks, different hardware, different power)

Limit: MAX 7 instances per AZ per placement group
Can span AZs

Use cases:
├── Small critical HA clusters (ZooKeeper, etcd)
├── Primary + standby databases
└── Any workload where simultaneous failure is unacceptable
```

---

3. Partition Placement Group

```
Goal: Balance between performance and resilience
      for large distributed systems

Physical layout:
┌──────────────┐  ┌──────────────┐  ┌──────────────┐
│ Partition 1  │  │ Partition 2  │  │ Partition 3  │
│ (own racks,  │  │ (own racks,  │  │ (own racks,  │
│  own power)  │  │  own power)  │  │  own power)  │
│  EC2 EC2 EC2 │  │  EC2 EC2 EC2 │  │  EC2 EC2 EC2 │
└──────────────┘  └──────────────┘  └──────────────┘

Up to 7 partitions per AZ
```

```
Hundreds of instances per partition
Instances in same partition may share hardware
Instances in different partitions NEVER share hardware

Partitions visible via instance metadata:
curl http://169.254.169.254/latest/meta-data/placement/partition-number

Use cases:
├── HDFS (put DataNodes in different partitions)
├── Cassandra (put replicas in different partitions)
└── Kafka (spread brokers across partitions)
```

💬 Short Interview Answer

"EC2 Placement Groups control physical placement of instances. Cluster packs instances onto the same rack for maximum bandwidth and minimum latency — used for HPC and ML training but all instances share the same failure domain. Spread puts each instance on separate hardware — maximum resilience, limited to 7 instances per AZ. Partition is the middle ground for large distributed systems like Cassandra or HDFS — you define partitions, each with their own racks and power, and place replica sets in different partitions so a hardware failure only affects one partition."

⚠ Gotchas

Gotcha	Detail
Cluster = one AZ only	Can't span AZs. If AZ goes down, everything goes down
Can't merge PGs	You can't merge two placement groups
Existing instances	You can't add an existing running instance to a PG — must stop it first
Capacity errors	Cluster PG can fail to launch if AWS can't fit all instances on adjacent hardware — launch all at once

---

2.7 EC2 Instance Metadata & IMDSv2

🟢 What It Is in Simple Terms

Every EC2 instance can ask "who am I?" by querying a special IP address: `169.254.169.254`. This returns data about the instance — its ID, AZ, IAM credentials, public IP, etc. IMDSv2 is the secure version that requires a session token first.

⚙️ How It Works

IMDSv1 — INSECURE (Legacy)

```
Simple GET request, no auth required:

curl http://169.254.169.254/latest/meta-data/instance-id
→ i-0abc123def456789

The SSRF attack problem:
┌──────────────────────────────────────────────────────────────┐
│ Attacker finds SSRF vulnerability in your web app            │
│ Sends request: GET http://169.254.169.254/latest/meta-data/  │
│                    iam/security-credentials/my-role          │
│ Gets back: AccessKeyId, SecretAccessKey, Token               │
│ Now attacker has full IAM role permissions!                  │
└──────────────────────────────────────────────────────────────┘
This is how the Capital One breach (2019) happened.
```

IMDSv2 — SECURE (Required)

```
Two-step process:

Step 1: Get a session token (PUT request required)
curl -X PUT "http://169.254.169.254/latest/api/token" \
     -H "X-aws-ec2-metadata-token-ttl-seconds: 21600"
→ Returns: TOKEN_VALUE

Step 2: Use token in all metadata requests
curl -H "X-aws-ec2-metadata-token: TOKEN_VALUE" \
     http://169.254.169.254/latest/meta-data/instance-id

Why SSRF can't exploit IMDSv2:
SSRF attacks typically follow redirects and use GET.
IMDSv2 requires a PUT first → SSRF libraries can't do this.
→ Attacker cannot get the token → cannot get credentials.
```

Useful Metadata Paths

```
BASE=http://169.254.169.254/latest
TOKEN=$(curl -X PUT "$BASE/api/token" -H "X-aws-ec2-metadata-token-ttl-
seconds: 21600")
H="X-aws-ec2-metadata-token: $TOKEN"

# Instance identity
curl -H "$H" $BASE/meta-data/instance-id
```

```
curl -H "$H" $BASE/meta-data/instance-type
curl -H "$H" $BASE/meta-data/placement/region
curl -H "$H" $BASE/meta-data/placement/availability-zone

# Networking
curl -H "$H" $BASE/meta-data/local-ipv4
curl -H "$H" $BASE/meta-data/public-ipv4

# IAM credentials (rotated automatically by AWS)
curl -H "$H" $BASE/meta-data/iam/security-credentials/my-role

# Spot interruption notice (check every 5s on Spot instances)
curl -H "$H" $BASE/meta-data/spot/termination-time
# Returns 404 if not interrupted, returns time if 2-min warning fired

# User data
curl -H "$H" $BASE/user-data
```

Enforce IMDSv2 in Launch Template

```
{
  "MetadataOptions": {
    "HttpTokens": "required",
    "HttpEndpoint": "enabled",
    "HttpPutResponseHopLimit": 1
  }
}
```

> ⚠ **Hop Limit Gotcha:** If you're running containers on EC2 and the container needs metadata, set `HttpPutResponseHopLimit: 2`. With limit 1, the PUT token request from inside a container doesn't reach the metadata service due to the extra network hop.

💬 Short Interview Answer

"EC2 instance metadata is available at 169.254.169.254 and lets an instance discover its own instance ID, AZ, IAM credentials, and more at runtime. IMDSv2 is the secure version — it requires a session token obtained via a PUT request before any metadata can be fetched. This prevents SSRF attacks from stealing IAM credentials, which is how the Capital One breach happened using the old IMDSv1. In production, we enforce IMDSv2 via the Launch Template's MetadataOptions and an SCP that denies launching instances without it."

⚠ Gotchas

Gotcha	Detail

Gotcha	Detail
Hop limit for ECS/K8s	Set to 2 if containers need metadata access
IMDSv2 enforcement	Set via Launch Template AND via SCP at org level
IAM creds auto-rotate	Credentials from metadata rotate before expiry. Never cache them long-term
Instance identity document	`/latest/dynamic/instance-identity/document` — signed JSON you can verify to prove you're on a real EC2

2.8 Spot Instances — Interruption Handling, Spot Fleets, Diversification

---

🟢 What It Is in Simple Terms

Spot instances are unused EC2 capacity that AWS sells at a steep discount. The catch: AWS can reclaim them with 2 minutes notice. Done right, Spot can power production workloads at a fraction of the cost.

⚙️ How Spot Works Internally

```
AWS has spare physical capacity in every AZ at all times.
Rather than leave it idle, AWS sells it as Spot.

Spot Price:
├── Changes based on supply/demand (but rarely spikes now)
├── Typically 70-90% below On-Demand
└── You pay current Spot price, not a "bid" (AWS changed this)

Interruption trigger:
├── AWS needs capacity back (someone launched On-Demand/Reserved)
├── Spot price exceeds your max price (if you set one)
└── AWS gives 2-minute warning via:
    ├── Instance metadata: /meta-data/spot/termination-time
    ├── EventBridge event: EC2 Spot Instance Interruption Warning
    └── CloudWatch event
```

🧩 Spot Fleet & EC2 Fleet

```
Problem: Relying on one instance type/AZ = gets interrupted together

Solution: Spot Fleet (diversification)

Spot Fleet config example:
{
  "TargetCapacity": 100,
  "AllocationStrategy": "capacity-optimized",
  "LaunchSpecifications": [
    {"InstanceType": "m5.xlarge",  "SubnetId": "subnet-az1"},
    {"InstanceType": "m5a.xlarge", "SubnetId": "subnet-az1"},
    {"InstanceType": "m6i.xlarge", "SubnetId": "subnet-az2"},
    {"InstanceType": "m4.xlarge",  "SubnetId": "subnet-az2"},
    {"InstanceType": "c5.2xlarge", "SubnetId": "subnet-az3"}
  ]
}

Allocation strategies:
├── capacity-optimized       → launch from pools with most available
│                              capacity (lowest interruption risk) ←
RECOMMENDED
├── lowest-price             → cheapest option (higher interruption risk)
├── diversified              → spread evenly across all pools
└── price-capacity-optimized → balance of price and capacity (newer)
```

🛡️ Interruption Handling Pattern

```bash
#!/bin/bash
# Spot interruption handler — run every 5 seconds on instance
while true; do
  TERMINATION=$(curl -sf \
    -H "X-aws-ec2-metadata-token: $TOKEN" \
    http://169.254.169.254/latest/meta-data/spot/termination-time \
    2>/dev/null)

  if [ -n "$TERMINATION" ]; then
    echo "Spot interruption at: $TERMINATION"

    # 1. Stop accepting new work
    # 2. Checkpoint current progress to S3
    # 3. Drain active connections
    # 4. Deregister from load balancer
    # 5. Complete ASG lifecycle hook

    aws autoscaling complete-lifecycle-action \
      --lifecycle-action-result CONTINUE \
      --instance-id $(curl -sf .../instance-id) \
      --lifecycle-hook-name spot-interruption-hook \
      --auto-scaling-group-name my-asg
```

```
    break
  fi
  sleep 5
done
```

🔧 ASG Mixed Instances Policy (Production Pattern)

```
{
  "AutoScalingGroupName": "my-mixed-asg",
  "MixedInstancesPolicy": {
    "InstancesDistribution": {
      "OnDemandBaseCapacity": 2,
      "OnDemandPercentageAboveBaseCapacity": 20,
      "SpotAllocationStrategy": "capacity-optimized"
    },
    "LaunchTemplate": {
      "LaunchTemplateSpecification": {
        "LaunchTemplateName": "my-lt",
        "Version": "$Latest"
      },
      "Overrides": [
        {"InstanceType": "m6i.xlarge"},
        {"InstanceType": "m6a.xlarge"},
        {"InstanceType": "m5.xlarge"},
        {"InstanceType": "m5a.xlarge"},
        {"InstanceType": "c6i.2xlarge"}
      ]
    }
  }
}
```

This keeps 2 On-Demand always running as a baseline, fills the rest 80% Spot / 20% On-Demand, across 5 instance types for maximum capacity pool diversification.

💬 Short Crisp Interview Answer

"Spot Instances are spare AWS capacity offered at 70–90% discount, but AWS can interrupt them with 2 minutes notice. The key to using Spot in production is diversification — never rely on a single instance type or AZ. Use ASG Mixed Instance Policies with 5+ instance types across multiple AZs. Use the capacity-optimized allocation strategy to minimize interruptions. Design your application to handle interruptions gracefully — checkpoint state to S3, drain connections on the 2-minute warning. For stateless workloads like web servers or batch processing, you can run 80% Spot with 20% On-Demand as baseline and get reliable, cheap compute."

🏭 Real World Production Example

```
ML Training Cluster at an AI startup:

Problem: Training jobs take 8-24 hours on p3.8xlarge instances.
         On-Demand cost: $12.24/hr × 20 instances × 16 hrs = $3,916

Solution:
├── Use Spot p3.8xlarge: ~$2.50/hr = $800 (80% savings!)
├── Checkpoint model weights to S3 every 30 minutes
├── If interrupted: job resumes from last checkpoint
├── Use 3 instance types: p3.8xlarge, p3.16xlarge, g4dn.12xlarge
│   (all equivalent GPU compute, spread across capacity pools)
├── Use capacity-optimized strategy
└── SQS job queue: interrupted job returns to queue, retried

Result:
├── Cost: $800 instead of $3,916
├── Average interruption: 1-2 per 16hr job
├── Max checkpoint loss: 30 minutes of training
└── Total extra time due to interruptions: ~1 hour
```

⚠ Gotchas

Gotcha	Detail
Spot ≠ unreliable	With proper diversification, interruption rates are low (< 5% per hour)
Spot for stateful apps	Only if you handle interruption perfectly — checkpoint and resume
EBS on Spot	EBS volumes survive Spot interruption if `DeleteOnTermination: false`. Data is safe
Spot blocks (deprecated)	Used to offer 1-6hr guaranteed slots. AWS deprecated this in 2021
Reserved + Spot myth	RI discount applies to On-Demand matching instances, not Spot
Spot in EKS	Use Karpenter or Cluster Autoscaler with Spot nodegroups — Karpenter is better at multi-type diversification

---

🔗 Category 2 — Full Connections Map

```
COMPUTE (EC2) connects to everything:

EC2 Instances
├── IAM Instance Profile    → AWS API permissions
```

```
├── VPC/Subnets/SGs          → Network isolation
├── EBS/EFS/Instance Store   → Storage
├── AMI                      → Boot image (Golden AMI pattern)
├── ASG                      → Fleet management + scaling
│   ├── Launch Templates     → Instance config (versioned)
│   ├── Scaling Policies     → Target tracking / step
│   ├── Lifecycle Hooks      → Custom launch/terminate logic
│   └── Instance Refresh     → Rolling AMI updates
├── ALB/NLB                  → Traffic distribution
│   └── Target Groups        → Health check + routing
├── Placement Groups         → Physical placement strategy
├── Spot Fleet               → Cost optimization + diversification
├── CloudWatch               → Metrics and alarms driving ASG
├── SSM                      → Patch, access, config
├── KMS                      → EBS encryption
└── IMDSv2                   → Secure metadata access
```

📌 Quick Reference — Interview Cheat Sheet

Topic	Key Number / Fact
Spot discount	70–90% off On-Demand
Spot warning	2 minutes before interruption
Savings Plans discount	Up to 66% off
ASG default cooldown	300 seconds
Spread PG limit	7 instances per AZ
Partition PG limit	7 partitions per AZ
User data size limit	16 KB
ALB idle timeout default	60 seconds
ALB deregistration delay	300 seconds default
IMDSv2 token TTL max	21600 seconds (6 hours)
NLB cross-zone LB	Off by default (costs extra)
ALB cross-zone LB	On by default
t3 CPU credit mode	Standard (throttle) or Unlimited (charged)
Launch Template versions	Unlimited, with Default and Latest pointers

---

Guide prepared for DevOps / SRE / Platform Engineer interview preparation.
Excluded topics: CodeBuild, CodePipeline, CloudFormation.